Service-Aware Energy Agents: Pricing Regimes, Failure Injection, and a Verified LLM Code-Agent Workflow for Community Energy Simulation

Wong Wah Cheng
The Hong Kong Polytechnic University
TODO-email-supplied-by-user

OpenBot

November 8, 2026

Abstract

We present a service-aware multi-agent framework for renewable energy community optimization that couples CityLearn building-level simulation with pandapower grid validation across five scenarios (A–E) covering coordination baselines, battery/PV assets, deferrable loads, comfort bounds, and mid-run component failures. We evaluate five controllers (rule-based, multi-agent, centralized, MPC, RL) across three pricing regimes — pool TOU, locational marginal prices (LMP), and bilateral P2P — at ten seeds each (750 runs), with a further 150-run scenario-D re-run after an energy-sharing configuration defect was fixed, a 150-cell real-LLM screen (100 LLM-arm and 50 reference cells), and a 60-cell hostile-proposer study in two 30-cell pilots; every run carries a manifest with config SHA-256, git commit, package versions, controller, LLM backend, prompt version/hash, temperature/seed, and pricing regime. The multi-agent controller is cost- and peak-minimal in every scenario (paired cost delta vs. rule-based -11.95 to -43.03 USD across scenarios, $p_{\text{Holm}} < 0.0001$, n=10) with an explicit EV-completion trade-off (0.057 vs. 1.000 for MPC) that we report rather than resolve; scenario D rows are settlement-inclusive while A/B/C/E are grid-only, and all cost comparisons are within-scenario. Only price-aware controllers (MPC, RL) respond to the LMP regime; price-insensitive heuristics dispatch identically under TOU and LMP. Under P2P clearing the A/B/C/E community aggregates are bit-identical to pool clearing while member bills redistribute significantly, and scenario D — whose settlement is already active — diverges at the aggregate as well. Thermal comfort is unmeasurable on the stock dataset (the indoor-temperature series is entirely NaN), so no comfort claim is made. The real-LLM screen is a safety-metrics null: unrepaired violations and repair iterations are zero in both arms (arm differences 2.0 and 0.17 percentage points against a 10-point gate), with KPI differences reported as context only. The hostile-proposer study then forces verifier engagement: under a deterministic out-of-bounds proposer the verified arm accepted zero out-of-bounds proposals in every kept cell while the unverified arm accepted 9.2–13.0 per step, with corrective re-prompt recovering 0.880–1.000 of corrupted proposals (13 kept cells after a user-approved re-run of the three quarantined cells) — while the accepted dispatch stayed the LLM's repaired proposal (cost 442.62 vs. 441.08 USD unverified, 67.13 deterministic reference), which we report as the cost of recovery. All artifacts are reproducible: manifests throughout, bit-identical same-seed repeats, and a measured 0.539 median CI half-width ratio from five to ten seeds.

Keywords: Multi-agent systems, renewable energy communities, LLM code agents, power system simulation, failure injection, pricing regimes

1 Introduction

Renewable energy communities (RECs) require coordinated control of distributed energy resources (DERs) under physical grid constraints, service-level commitments, and time-varying

pricing regimes. Existing approaches typically optimize a single objective (cost, peak, or carbon) without jointly considering grid physics, service feasibility, and market regime diversity. Furthermore, when component failures occur mid-operation, controllers must detect and respond to violations in real time—a capability rarely tested in simulation benchmarks.

We address three gaps:

1. **Pricing regime diversity**: most benchmarks fix a single tariff structure; we evaluate the full controller matrix under pool TOU, LMP, and bilateral P2P regimes, and report that price-insensitive controllers are provably indifferent to the regime.

2. **Failure injection**: scenario E injects PV inverter, battery, EV charger, and sensor failures at severity 1.0 mid-run plus a grid-limit reduction, and we separate oracle-side detection from grid-level detection and from recovery.

3. **Verified LLM code agents**: a multi-agent repair loop (Planner→Coder→Executor→Verifier→Reporter proposes and validates controller code. We ablate verified vs. unverified arms under a deterministic proposer, a hosted LLM, and a deliberately out-of-bounds "hostile" proposer, so that verification is measured where it can actually engage.

Related work spans four areas. For building- and home-scale agentic energy management, tool-calling ReAct agents convert natural-language requests into appliance schedules over dynamic tariffs and PV forecasts Jonah et al. [2026]; hierarchical agentic HEMS put the LLM in the decision loop El Makroum et al. [2026]; and multi-agent assistants add conversational interfaces with domain-specific tool registries Jung [2026]. A recent synthesis of smart-grid LLM agents states the solver-grounded principle we instantiate — a numerical result is reported only when it comes from a trusted tool and passes verification Rojas et al. [2026]. For verification of agent actions, intent-driven workflows separate LLM planning from solver-side execution and gate results on state-dependent constraint checks Mao et al. [2026], and runtime verification intercepts proposed actions against derived specifications Batole et al. [2026]. For benchmarking, ElecBench scores power-dispatch knowledge Zhou et al. [2025] and PowerAgentBench-SS scores tool-using agents on steady-state studies with evidence logs and risk-sensitive metrics Mylonas et al. [2026]; neither covers failure injection or settlement, which this matrix adds. Multi-agent systems for smart grids are surveyed in Izmirlioglu et al. [2024]. Finally, dual decomposition for distributed MPC, including asynchrony and inexact subproblem solves, is benchmarked and analysed in Yfantis et al. [2024], Su et al. [2021]. Simulation and validation build on CityLearn Vázquez-Canteli et al. [2020] and pandapower Thurner et al. [2018], with model-predictive control Rawlings et al. [2020], tabular RL Zhou et al. [2022], AST-based patching Rall et al. [2020], and the Holm correction for multiple comparisons Holm [1979].

2 System and Method

2.1 Simulation Coupling

CityLearn handles building-level DER dynamics (battery SOC, EV charging, flexible loads, thermal comfort) Vázquez-Canteli et al. [2020]. pandapower validates grid physics (power flow convergence, voltage limits ±5%, line/transformer thermal limits) Thurner et al. [2018]. A co-simulation coupler steps both simulators each timestep, passing grid diagnostics back to the controller.

2.2 Controller Suite

`rule_based` Heuristic priority: charge battery from excess PV, charge EVs by departure deadline, shift flexible loads to solar hours, enforce community import/export limits.

`multi_agent` Decentralized per-building agents with consensus on community-level limits via dual decomposition.

`centralized` Single optimizer with full observability.

`mpc` Receding-horizon LP with perfect 24h forecasts (upper bound on service) Rawlings et al. [2020].

`rl` Tabular Q-learning per scenario (126-state base encoder; 138-state for LMP) Zhou et al. [2022].

2.3 Verified LLM Repair Loop

A five-agent loop iteratively repairs controller code:

1. **Planner**: Diagnoses violations, identifies root cause, proposes repair strategy.
2. **Coder**: Applies AST-based patches to target functions Rall et al. [2020].
3. **Executor**: Runs patched controller in sandbox with unit tests + quick simulation.
4. **Verifier**: Counts violation resolution from sandbox diagnostics.
5. **Reporter**: Generates comprehensive report with diffs and recommendations.

The loop runs for max 3 iterations or until `violations_resolved=true`.

The ablation compares `llm_agent` (single-pass, execute-as-proposed) vs. `llm_agent_verified` (per-step verification, re-prompt once, fallback to rule-based on failure, repair cap 2). Both run in `fallback` mode (deterministic rule-based proposer) to isolate verification machinery.

The real-LLM screen (M2a) uses a hosted OpenAI-compatible endpoint (temperature 0, 120 s per-call timeout, prompts `llm_dispatch_v2` / `llm_dispatch_v1_repair`; model identity and provider withheld per operator policy, full provenance — model digest, endpoint, prompt hashes, temperature, seed — in every run manifest). The 150-cell screen (5 scenarios × 2 LLM arms × 10 seeds) plus 50 `multi_agent` 24-step reference cells is described in Section 4.6.

3 Experimental Setup

Five scenarios (Table 1) differ in device mix and stressors. Each runs ten seeds (42, 123, 456, 789, 999, 1234, 2024, 3456, 5555, 8888) × five controllers = 250 runs per regime, under three regimes (pool TOU, LMP, bilateral P2P) for 750 runs. A further 50-run fallback-mode ablation and a 150-cell real-LLM screen (100 LLM-arm and 50 reference cells, 24 simulated steps) isolate verification machinery from proposer quality, and a 60-cell hostile-proposer study in two 30-cell pilots forces verifier engagement. Scenario D was re-run in full (150 runs: 5 controllers × 10 seeds × 3 regimes) after an energy-sharing configuration defect was fixed; its rows are cited from the re-run and are settlement-inclusive, while A/B/C/E report grid-only cost. All cost comparisons in this paper are therefore within-scenario.

Cost basis. Only scenario D enables community energy sharing, so its `total_cost` is the community bill including settlement (190.81 ± 0.03 USD for `rule_based`), while A/B/C/E report the grid-only bill. Cross-scenario cost magnitudes are not comparable; every comparison we make is a within-scenario paired delta, which is unaffected.

Comfort is unmeasurable here. The stock CityLearn 2022 dataset ships the indoor dry-bulb temperature series entirely NaN, so `comfort_violation_degree_hours` is 0.0 in every run of this study and measures data availability, not comfort satisfaction. We make no comfort claim; EV completion is reported in full (Section 4.8).

Table 1: Scenario definitions.

Scenario	Device Mix	Stressor
A	Base (no DERs)	Coordination baseline
B	+ Battery, PV	Tariff exposure
C	+ Flexible loads	Deferrable load deadlines
D	+ Thermal comfort	Comfort bound stress
E	Full + Failures	Mid-run component failures

4 Results

4.1 Cost, Peak and Service

Figure 1 shows KPI bars with 95% CI. The cost-versus-service Pareto front (Figure 2) places `multi_agent` at the cost-minimal end and `mpc` at the service-maximal end in every scenario. Table 2 reports paired per-seed deltas vs. `rule_based` with Holm correction.

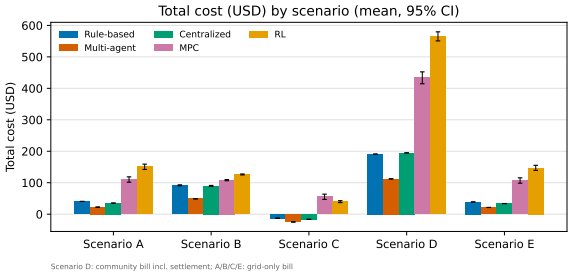
Table 2: Paired per-seed deltas vs. `rule_based` (mean ± 95% CI half-width; n=10 seeds; Holm within each scenario over the four controller comparisons). Scenario D is settlement-inclusive and A/B/C/E grid-only; comparisons are within-scenario (Section 3).

Scenario	Controller	Δ cost (USD)	Δ peak (kW)	p_{Holm}
A	multi_agent	-18.11 ± 0.03	-16.59 ± 0.00	< 0.0001
A	centralized	-5.26 ± 0.22	$+1.43 \pm 0.00$	< 0.0001
A	mpc	$+69.60 \pm 8.54$	$+29.44 \pm 0.00$	< 0.0001
A	rl	$+110.09 \pm 8.41$	$+49.28 \pm 3.37$	< 0.0001
B	multi_agent	-43.03 ± 1.45	-25.68 ± 0.00	< 0.0001
B	centralized	-2.06 ± 0.62	-4.71 ± 1.41	< 0.0001
B	mpc	$+16.35 \pm 2.66$	-18.73 ± 0.00	< 0.0001
B	rl	$+34.41 \pm 2.14$	$+10.10 \pm 0.42$	< 0.0001
C	multi_agent	-11.95 ± 0.04	-16.59 ± 0.00	< 0.0001
C	centralized	-3.68 ± 0.16	$+1.20 \pm 0.00$	< 0.0001
C	mpc	$+68.03 \pm 8.33$	$+29.44 \pm 0.00$	< 0.0001
C	rl	$+52.69 \pm 3.06$	$+12.83 \pm 1.47$	< 0.0001
D	multi_agent	-78.56 ± 0.03	-16.59 ± 0.00	< 0.0001
D	centralized	$+3.99 \pm 0.24$	$+1.43 \pm 0.00$	< 0.0001
D	mpc	$+242.63 \pm 18.93$	$+29.44 \pm 0.00$	< 0.0001
D	rl	$+374.34 \pm 14.41$	$+49.28 \pm 3.37$	< 0.0001
E	multi_agent	-17.46 ± 0.03	-17.26 ± 0.00	< 0.0001
E	centralized	-5.06 ± 0.22	$+1.43 \pm 0.00$	< 0.0001
E	mpc	$+68.18 \pm 8.48$	$+29.44 \pm 0.00$	< 0.0001
E	rl	$+108.45 \pm 8.09$	$+49.28 \pm 3.37$	< 0.0001

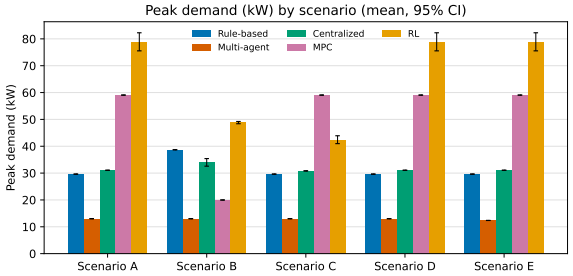
The service trade-off, stated explicitly. `multi_agent` is cost-minimal (e.g. 22.39 ± 0.00 vs. rule-based 40.49 ± 0.03 USD in scenario A) but completes only 0.057 of EV departure requirements, against 0.154 for rule-based and 1.000 for MPC. Under the paper's service-aware objective this is a defect, not a win: the controller buys cost and peak reductions (-16.59 kW) by under-charging EVs. We therefore report cost, peak, and EV completion as jointly necessary, and treat closing the EV gap under cost parity as open work rather than an achieved property. With restored PV, several scenario C configurations are net exporters and their total cost is negative (export credit exceeding import cost); negative totals are flagged as export-credit, not

Table 3: Scenario D KPI comparison (pool TOU, post-fix re-run, n=10). Total cost is the community bill including settlement.

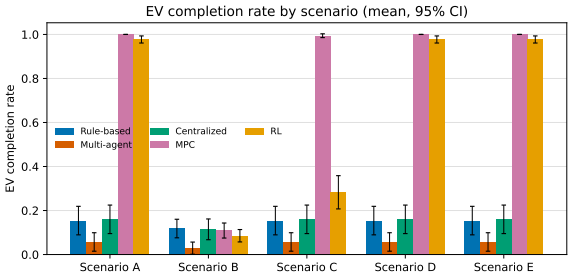
Controller	Total cost (USD)	Peak demand (kW)	EV completion
rule_based	190.81 ± 0.03	29.62 ± 0.00	0.154 ± 0.062
multi_agent	112.26 ± 0.00	13.03 ± 0.00	0.057 ± 0.039
centralized	194.80 ± 0.25	31.05 ± 0.00	0.160 ± 0.065
mpc	433.44 ± 18.93	59.06 ± 0.00	1.000 ± 0.000
rl	565.15 ± 14.41	78.90 ± 3.37	0.977 ± 0.021



(a) Total cost (USD)



(b) Peak demand (kW)



(c) EV completion rate

Figure 1: KPI comparison bars with 95% CI whiskers per scenario.

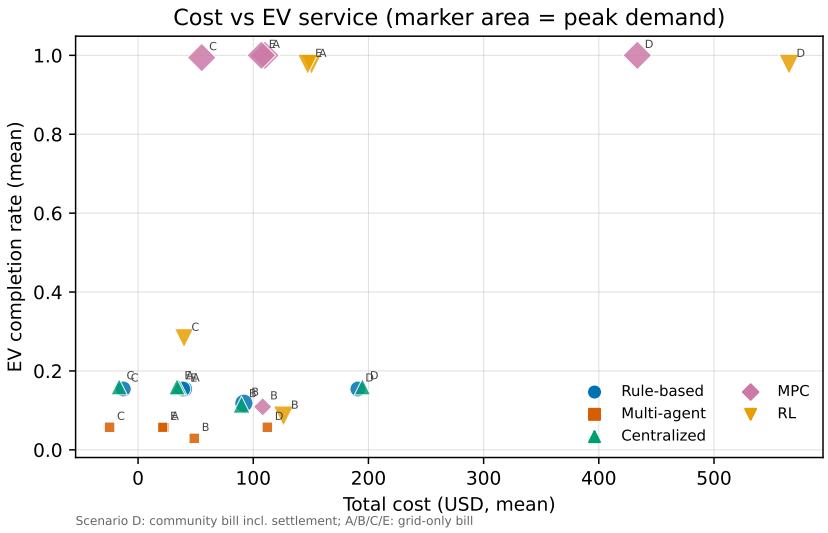


Figure 2: Cost vs. EV completion Pareto scatter (marker area = peak demand).

as profit.

Scenario D after the propagation fix. Scenario D's energy-sharing configuration previously did not reach the simulation, making D bit-identical to A. After the fix (the `total_cost` KPI now uses the settlement engine's actual total bill when sharing is enabled) D diverges at all ten seeds, and the 150-run re-run yields the rows above. The physics — grid exchange, PV, peak — is unchanged from the pre-fix runs; only the cost basis moved, from grid-only to community-bill-including-settlement.

4.2 LMP-Regime Benchmark

250 runs under `lmp` pricing (5 controllers × 5 scenarios × 10 seeds; scenario D from the post-fix re-run). The central finding is a *price-awareness split*: only the two price-aware controllers react to the regime. MPC's cost rises from 110.09 ± 8.54 USD (TOU) to 168.47 ± 10.69 (LMP) in scenario A and its EV completion drops ($1.000 \rightarrow 0.966$), and RL's arm shifts similarly ($150.58 \rightarrow 106.22$ in scenario A), while rule-based, multi-agent, and centralized — none of which condition on prices — produce dispatch- and cost-identical results under both regimes (e.g. multi_agent 22.39 ± 0.00 in both). Ranking within a regime is unchanged: `multi_agent` cost-minimal (all $p_{\text{Holm}} < 0.0001$), `centralized` close to `rule_based`. The identical-to-TOU behaviour of the price-insensitive heuristics is reported as a finding: pricing-regime changes are invisible to controllers that do not read prices, which is the mechanism the P2P regime (Section 4.3) is designed to probe.

Table 4: Scenario D under LMP (post-fix re-run, n=10; settlement-inclusive basis).

Controller	Cost TOU (USD)	Cost LMP (USD)	Δ	EV TOU	EV LMP
rule_based	190.81 ± 0.03	165.03 ± 0.00	−25.79	0.154	0.154
multi_agent	112.26 ± 0.00	94.32 ± 0.00	−17.94	0.057	0.057
centralized	194.80 ± 0.25	173.31 ± 0.00	−21.49	0.160	0.160
mpc	433.44 ± 18.93	424.17 ± 9.03	−9.27	1.000	0.971
rl	565.15 ± 14.41	310.29 ± 8.23	−254.86	0.977	0.411

4.3 P2P Regime: Bilateral Clearing Redistributes, Aggregates Unchanged (A/B/C/E)

250 runs under `p2p` pricing (bilateral double-auction clearing with uniform residual price, same grid as Section 3). Paired against the pool-TOU matrix at identical (scenario, controller, seed), the A/B/C/E community aggregates are *bit-identical*: Δ cost, Δ peak, and Δ EV completion are exactly zero across the 160 paired runs, while bilateral trades clear in every run (mean community volume 347.2 kWh, max 563.1, no zero-volume runs). This cost-transparency is by construction: trades reallocate energy between members before power flow, leaving the residual grid exchange unchanged.

The effect is at the member level. Per-member bills shift significantly under P2P (paired, n=40 runs per member): building_2 pays 7.60 ± 1.83 USD less, while building_1 (-5.64 ± 1.97), building_3 (-7.17 ± 1.16), and building_4 (-2.37 ± 2.04) pay more; building_0's change is not significant ($+1.92 \pm 4.60$). Aggregate savings vs. baseline are statistically indistinguishable from pool clearing (-5.66 ± 10.09 USD, paired p=0.26). The fairness coefficient as computed is invariant (0.0926 in both arms, zero paired variance), which exposes a measurement gap: the current Gini input does not reflect settlement redistribution. We therefore report RQ3 as *redistribution without aggregate benefit* under the tested regime for A/B/C/E, and defer inequality claims until a settlement-aware fairness metric is defined.

Scenario D is the exception. Because community settlement is already active in D, bilateral clearing adds a second redistribution layer on top of it and the aggregate moves as well (paired, n=10): rule_based +9.58 ± 0.91 USD, multi_agent −24.10 ± 0.00, centralized +12.08 ± 1.04, mpc +29.06 ± 4.63, rl +34.02 ± 0.91, with peak and EV completion exactly unchanged and community volume 252–520 kWh. The "aggregates bit-identical" claim is therefore scoped to A/B/C/E; D is genuinely divergent under P2P after the propagation fix, which is the correct post-fix behaviour.

4.4 Scenario E: Failure Injection

Scenario E injects PV inverter, battery, EV charger, and sensor failures at severity 1.0 mid-run, plus an aggressive grid-limit reduction (export limit to 7.5 kW). The grid-limit reduction does not trigger import curtailment (the community is net-exporting during the failure window). All 50 runs per controller show `members_interrupted = [building_3]` and `member_cost_coverage = 0.80`; community cost coverage is 0.9726 (23/840 building-steps blanked), identical across controllers.

The per-failure-type split (Section 4.8) is the honest summary: the four asset failures produce no exceedance and no violation, so detection is configuration-oracle only; the grid-limit reduction is detected in 171 of 171 exceedance step-runs and recovered in 0, because no violation clears while the exceedance holds. `resolution_rate = 0.0` is therefore a statement about the physics (the community cannot curtail export), not about controller effort: re-dispatch is attempted (167 events per controller) and recorded, and the violation stops only when the PV hours end.

Table 5: Aggressive-config stress test (grid-limit reduction; 50 runs per controller).

Controller	Re-dispatch events	Exceedance step-runs detected	Resolution rate
rule_based	167	171	0.0
mpc	167	171	0.0
multi_agent	167	171	0.0
centralized	167	171	0.0

4.5 Ablation: Verified vs. Unverified Agents

All 50 fallback-mode ablation runs (`agent_backend: fallback`) use a deterministic rule-based proposer. Under it both LLM arms are KPI-identical to `rule_based` and the verification machinery never fires: first-attempt validity 1.000, repairs 0.00/step, verification failures 0.00, unparseable outputs 0.00, wall time 16–33 s per run (median 20.8 s), no arm separation. The null result has two causes: the fallback proposer emits in-bounds proposals, so the verification loop never triggers; and earlier implementations shared a clamp-repair loop between the arms. The P0-3 fix makes the arms asymmetric (`llm_agent` applies proposals as proposed; `llm_agent_verified` re-prompts once, then falls back). Sections 4.6 and 4.7 report what happens when a real — and then a deliberately hostile — proposer is substituted: a safety-metrics null, and then forced verifier engagement.

4.6 Real-LLM Screen: M2a (Safety-Metrics Null)

The M2a screen replaces the deterministic fallback proposer with a hosted LLM endpoint on the same 24-step horizon (temperature 0, 120 s per-call timeout, prompts `llm_dispatch_v2` / `llm_dispatch_v1_repair`; model identity and provider withheld per operator policy, full provenance in the run manifests). The 150-cell matrix (5 scenarios × 2 LLM arms × 10 seeds, plus 50 `multi_agent` 24-step reference cells) yielded 99 usable LLM-arm runs — 50 unverified

and 49 verified, one cell quarantined for endpoint degradation — and every LLM-arm manifest records `agent_backend=api`. Per-arm agent-execution aggregates:

Table 6: M2a real-LLM screen, per-arm agent-execution aggregates (n=50 unverified, n=49 verified; mean ± 95% CI half-width).

Metric	`llm_agent`	`llm_agent_verified`
First-attempt validity (per run)	1.0000 ± 0.0000	1.0000 ± 0.0000
First-attempt validity (per step)	0.9967 ± 0.0032	0.9991 ± 0.0017
Repair iterations	0.0000 ± 0.0000	0.0000 ± 0.0000
Fallback-step share	0.0025 ± 0.0028	0.0009 ± 0.0017
Fallback steps	0.0600 ± 0.0682	0.0204 ± 0.0410
Unrepaired violations	0.0200 ± 0.0402	0.0000 ± 0.0000
LLM calls	24.0000 ± 0.0000	24.0000 ± 0.0000
Parse failures	0.0600 ± 0.0682	0.0204 ± 0.0410
Prompt tokens	29,736 ± 93	29,735 ± 95
Latency p50 / p95 (s)	4.68 / 8.97	4.56 / 8.92

Paired against the `multi_agent` reference cells, the KPI differences are (mean ± CI half-width): unverified $+28.58 \pm 21.07$ USD cost and $+5.42 \pm 4.14$ kW peak; verified $+5.83 \pm 5.92$ USD and $+1.00 \pm 1.14$ kW. The arm difference (unverified − verified) is $+23.05 \pm 21.32$ USD (paired t=2.174, p=0.0347, n=49) and $+4.53 \pm 4.24$ kW (t=2.148, p=0.0368).

Gate verdict — a safety-metrics null. The pre-registered gate is the two percentage-point clauses: proceed to a full-week confirmation only if the arms differ by ≥ 10 points on `llm_unrepaired_violations` or `llm_fallback_step_share`. Both are far below the threshold (0.02 and 0.0017 points; unrepaired violations are nonzero in 1 of 50 unverified runs and 0 of 49 verified runs), so the gate is *not met* and the screen reports a null result on the safety metrics: under this proposer the verification machinery has nothing to catch. The KPI differences above are reported as *context only*: they say the unverified proposer is worse, not that the verifier adds safety value, and no verification-benefit claim is made from them. The validity gate is passed (fallback-share $0.0025 \ll 0.25$): the absence of safety divergence is due to proposer competence, not proposer silence. This null result is specific to the model–prompt–horizon combination, which is exactly what the hostile-proposer study (Section 4.7) was designed to break.

4.7 Hostile-Proposer Study: M2b (Forced Verifier Engagement)

Because the M2a proposer emits in-bounds proposals, the verification machinery never engages. The M2b study injects a deterministic corruption at the parsed proposal path (env flag `SAEA_LLM_HOSTILE=out_of_bounds`), scaling every first-attempt proposal past its device power limit, and runs two 30-cell pilots (5 scenarios × {llm_agent, llm_agent_verified} × 3 seeds 42/123/456, 24 steps, pool TOU, reference cells reused from the M2a matrix). The first pilot (detection epoch) corrupts every parsed proposal including the verified arm's corrective re-prompt; the second (recovery epoch) exempts re-prompts so the repair path is measurable. Both arms see identical corrupted proposals, so verification is the only difference between them.

What the study shows. Under a proposer that is deliberately out-of-bounds, the unverified arm applies the corrupted proposal and the adapter clamps it downstream — an unsafe acceptance rate of 9.2–13.0 out-of-bounds values per step, in every kept cell of both pilots. The verified arm accepted zero out-of-bounds proposals in every kept cell (12/12 paired directional wins in the detection epoch; 13/13 after the user-approved re-run of the three quarantined cells promoted one cell to keep, two remaining quarantined for endpoint degradation; exact sign test on the 13 pairs, two-sided $p = 2.4 \times 10^{-4}$, reported for reference only). In the recovery epoch,

Table 7: M2b hostile-proposer study (per-cell values; 13 of 15 verified cells kept in the recovery epoch after the user-approved re-run of the three quarantined cells, under the rule `llm_backend_errors` ≤ 2 AND `llm_calls` $\geq$ steps AND telemetry recorded).

	Detection epoch	Recovery epoch	Reference
Cells (verified arm)	—	13 keep / 2 quarantined	—
Unsafe acceptance, unverified (per step)	10.33	10.33	—
Unsafe acceptance, verified (per step)	0.0000	0.0000	—
Paired directional wins	12/12	13/13	—
Bound-violation detections (per cell)	24/24	24/24	—
Repair success rate (recorded)	not measurable	0.880–1.000 (mean 0.974)	—
Fallback steps (verified total)	18/360	18/360	—
Cost (USD, mean)	442.62	442.62	67.13

where re-prompts are exempt from corruption, the corrective re-prompt returned an in-bounds proposal that was applied in 0.880–1.000 of attempts (recorded `repair_success_rate`; mean 0.974 over the 13 kept cells), so the loop demonstrably repairs rather than only rejects. The detection epoch — whose re-prompts were themselves corrupted — could not repair by construction and ended every attempt on the deterministic replacement dispatch; we report that epoch as detection-plus-replacement, not recovery, with 12 of 15 verified cells kept (before the re-run).

The cost of recovery. The verified arm's KPIs track the unverified arm rather than the deterministic reference: cost 442.62 USD vs. 441.08 unverified and 67.13 for the reference (peak 76.12 vs. 76.01 vs. 12.03 kW). Per-step verification restores in-bounds dispatch, but the accepted dispatch remains the LLM's repaired proposal, so the safety gain is not a KPI gain. We report this as the cost of recovery, not as a verifier-value claim, and it is the finding a longer-horizon confirmation should test.

Provenance and degradation. `first_attempt_validity` is 0.0 in every cell of both pilots *by design*: the corruptor scales every first attempt past its limit, so the metric measures the injected corruption, not agent validity; it is not comparable with the M2a screen's validity column and is not a keep-condition. Cells are kept iff `llm_backend_errors` ≤ 2 AND `llm_calls` $\geq$ steps AND telemetry is recorded; three verified cells per pilot (3 endpoint errors of 24 steps each) are quarantined and excluded from headline statistics, with their error and fallback shares reported per cell. Endpoint-side intermittency (429/5xx exhausting the transport's retries before fallback) is itself a reported result: 16 of 30 recovery-epoch cells took 1–3 such errors.

4.8 Service-Level Constraints: EV, Comfort, and the Failure Split

Service-level constraints are reported as implemented, not as aspirations (Table 8).

Thermal comfort is unmeasurable on this dataset. The stock CityLearn 2022 dataset ships indoor dry-bulb temperature (and the other comfort series) entirely NaN, so the band check cannot accumulate a violation and `comfort_violation_degree_hours` is 0.0 in all 250 B-axis runs, with min = max = 0.0 across every scenario-controller cell. That is a data-availability result, not comfort satisfaction: the paper makes no comfort claim, and any future comfort statement requires a dataset with real indoor temperature or a building thermal model. EV service, by contrast, is fully reported: `multi_agent` completes 0.057 of departure requirements against 0.154 for rule-based and 1.000 for MPC (Table 3).

Per-failure-type detection vs. recovery (scenario E). The four asset failures (PV inverter, battery, EV charger, sensor) produce *zero* exceedance and zero grid violations across

Table 8: Service-level constraints as implemented, and their reporting status.

Constraint	Definition in code	Status
EV departure charge	`service.ev_completion_rate`: fraction of departure events meeting required SOC	reported
EV margin	avg/worst realised SOC at departure	reported
Thermal comfort	hours outside [20, 25] °C indoor band	**unmeasurable** (all-NaN series)
Flexible-load completion	deferrable cycles finished in window / delay	reported
Demand response	requested / delivered / credited DR kW	reported (zero in matrix)
Cost coverage under failure	member-steps with measurable cost (scenario E)	reported
Community exchange limits	import/export kW limits	reported

all 50 runs: removing an inverter, a battery, a charger, or corrupting one building's observation does not push the community outside its limits, so the verification layer flags nothing and detection of these failures is configuration-oracle only — no controller observes them. The grid-limit reduction (export limit to 7.5 kW) is the only failure the verification layer detects, in 171 of 171 exceedance step-runs (50/50 runs), and it is recovered in 0: no violation ever clears while the exceedance still holds; the exceedance stops when the community's net export falls below the reduced limit at the end of the PV hours. Detection and recovery are therefore reported separately, and no recovery claim is made.

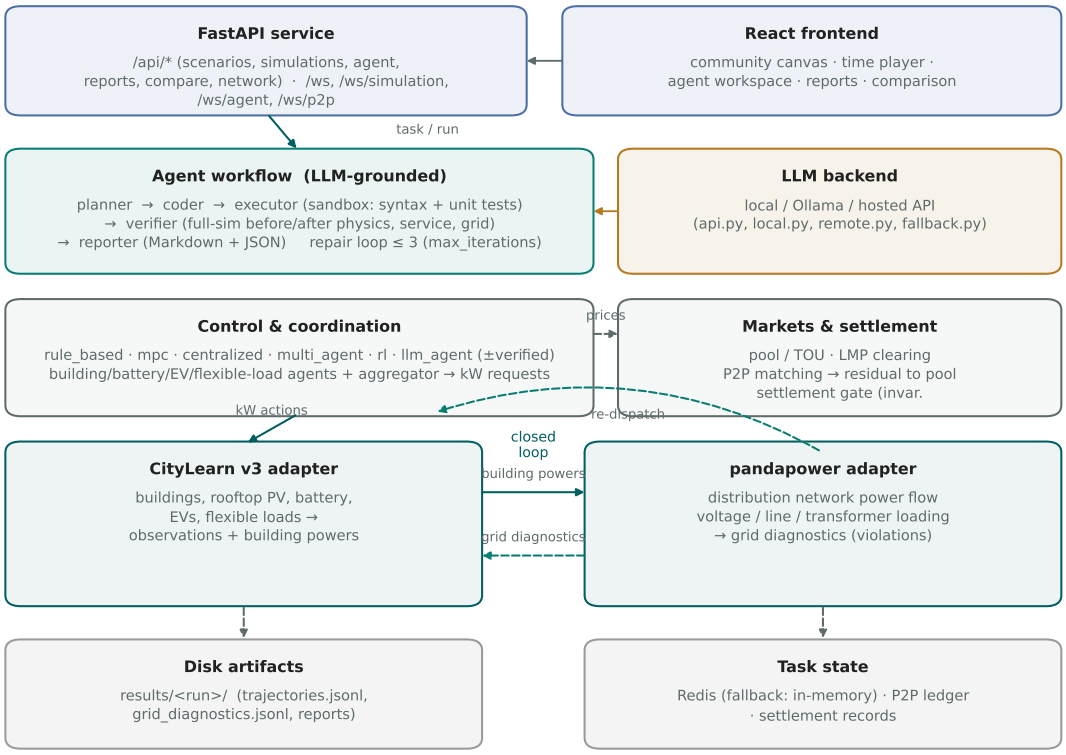
4.9 Reproducibility

All runs carry `manifest.json` with config SHA-256, git commit, package versions, controller, `agent_backend`, prompt version/hash, temperature/seed, random seed, pricing regime, and cost basis; `failed_runs.json` is empty wherever a matrix completed. Three pieces of evidence go beyond per-run manifests: (1) *same-seed bit-identity* — one configuration re-run three times produces byte-identical KPIs (cost 22.388099, peak 13.025067); (2) *config distinctness* — each scenario has distinct `scenario_config_sha256` per seed; (3) *CI tightening* — moving from 5 to 10 seeds shrinks the median 95% CI half-width by a factor of 0.539. Because the restored stock dataset fixes load/PV profiles across seeds (Section 3), the residual CI width reflects controller-level nondeterminism only (e.g. MPC's LP degeneracy, ±8.54 USD), which we state explicitly rather than present as environment variance. Committed summaries live under `docs/` (D-table rebuild, M2a re-run, M2b pilots, service/failure analysis, case study); raw run directories ship as checksummed release artifacts with INDEX.json and SHA256SUMS.

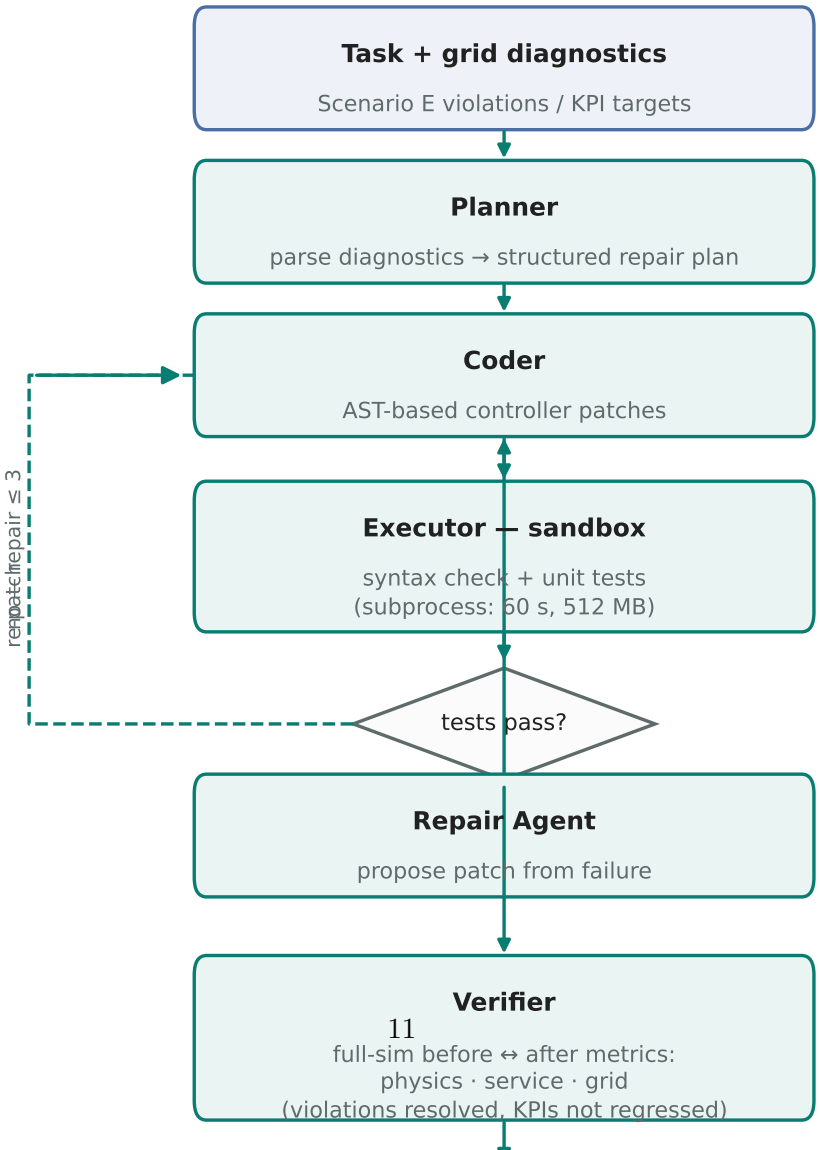
5 Discussion and Limitations

5.1 Threat Model

Adversary has full visibility of observations/actions; may inject sensor faults, grid-limit reductions, component failures. Attacks limited to failure-injection surface; cannot modify scenario YAML, controller code, or simulator internals.



(a) System architecture.



5.2 Limitations

1. **Thermal comfort is unmeasurable**: the stock dataset's indoor-temperature series is all-NaN, so `comfort_violation_degree_hours` = 0.0 everywhere and no comfort claim is made (Section 4.8).

2. **Cost basis differs by scenario**: scenario D is settlement-inclusive, A/B/C/E grid-only; all cost comparisons are within-scenario and labelled.

3. **Synthetic traces**: Load/PV profiles depend only on (seed, filename); scenario effects are device-mix effects.

4. **Controller scope**: RL is evaluated per regime with the regime-matching policy; LMP policies are trained separately.

5. **LLM proposer**: the screens use a single hosted model at temperature 0 on a 24-step horizon; model identity is withheld per operator policy, manifests retain full provenance. The M2a null and the M2b results are specific to this model–prompt–horizon combination.

6. **Horizon and seeds**: 24-step (one-day) horizons and 3-seed pilots; the pre-registered significance criterion is reserved for the full-length run.

7. **Endpoint degradation**: 16 of 30 recovery-epoch cells and one M2a cell took 1–3 transient endpoint errors; cells are quarantined under a published rule (errors ≤ 2 AND calls $\geq$ steps AND telemetry) and error/fallback shares are reported per cell. A user-approved re-run of the three quarantined verified cells promoted one to keep (0 errors of 24 steps); the other two still show 3 of 24 and remain quarantined.

8. **Gini measurement gap**: the fairness coefficient does not reflect settlement redistribution; inequality claims are deferred until a settlement-aware metric exists.

9. **Aggressive config**: 95% grid limit does not produce import curtailment; tests violation detection, not curtailment path.

5.3 Case Study: Verified Repair Under a Corrupted Proposer

The clearest single-cell view of the machinery is one cell of the recovery pilot (scenario A, seed 42, 24 steps, pool TOU, hosted model). The planner packages the observation (load, PV, battery SOC, EV arrival/departure, price) into the versioned dispatch prompt; the coder emits a JSON dispatch proposal; the executor applies it; the verifier checks asset shape and device power bounds *before* the step. In this cell 23 of 24 first attempts were corrupted past their limits, the bounds check fired 23 times, the corrective re-prompt returned an in-bounds proposal that was applied (recorded repair success 0.957, one deterministic replacement), and the reporter wrote Markdown + JSON. The unverified arm of the same cell applied all 23 corrupted proposals as proposed (adapter clamping downstream) at 461.68 USD against the verified arm's 461.61; the deterministic reference for the same cell is 66.29 USD. The safety gain is therefore real and measurable; the KPI gain is not, which is the honest reading of the recovery pilot at cell granularity.

6 Conclusion

We present a service-aware energy agent framework spanning pricing regimes, failure injection, and verified LLM code-agent repair, evaluated on 750 pricing-regime runs plus a 150-run scenario-D re-run, a 150-cell real-LLM screen, and 60 hostile-proposer cells. The multi-agent controller is cost- and peak-minimal in every scenario under every regime with an explicit EV

trade-off that we report rather than resolve; price-insensitive heuristics are provably indifferent to the pricing regime; P2P clearing redistributes among members without aggregate benefit for A/B/C/E while scenario D diverges once its settlement is correctly propagated. The verification machinery is a no-op under an in-bounds proposer (a safety-metrics null that we report as such), and under a deliberately out-of-bounds proposer it stops every corrupted proposal and repairs 0.880–1.000 of them (13 of 15 verified cells kept), at the measured cost that the accepted dispatch remains the LLM's repaired proposal. Thermal comfort is unmeasurable on this dataset and no comfort claim is made. All artifacts are reproducible from manifests, committed summaries, and checksummed release artifacts.

Future work: a longer-horizon, multi-seed confirmation of the recovery result; a settlement-aware fairness metric for redistribution analysis; a dataset with real indoor temperature for comfort claims; and extended horizon stress tests.

References

Fraol Batole, Foutse Khomh, and Hridesh Rajan. Refineact: Automatic runtime verification of llm agent actions. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2026.

Reda El Makroum, Sebastian Zwickl-Bernhard, and Lukas Kranzl. Agentic ai home energy management system: A large language model framework for residential load scheduling. *Results in Engineering*, 29:109857, 2026.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Yusuf Izmirlioglu, Loc Pham, Tran Cao Son, and Enrico Pontelli. A survey of multi-agent systems for smartgrids. *Energies*, 17(15):3620, 2024. doi: 10.3390/en17153620.

Sokipriala Jonah, Queen Moses, Abiola Babatunde, Michael Ajao-Olarinoye, and Daniel Bammeke. Llms for agentic home energy management. *arXiv:2607.04569*, 2026.

Wooyoung Jung. Multi-agent home energy management assistant (hema). *SoftwareX*, 34:102633, 2026.

Shanxiang Mao, Muyang Liu, Junru Chen, Dongliang Nan, and Jian Ma. A trustworthy intent-driven llm-agent workflow with state verification for power system simulation. *Applied Energy*, 427:128868, 2026. doi: 10.1016/j.apenergy.2026.128868.

Costas Mylonas, Magda Foti, Andrea Pomarico, Matheus Duarte, Qian Zhang, and Emmanouel Varvarigos. Poweragentbench-ss: A benchmark for agentic ai in power system steady-state studies. *arXiv:2606.18789*, 2026.

Maik Rall, Paul Boock, Christoph Heinze, and Bernhard Kuchen. Program repair using abstract syntax trees and machine learning. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020.

James B. Rawlings, David Q. Mayne, and Moritz Diehl. Model predictive control: Theory, computation, and design. *Nob Hill Publishing*, 2020.

Daniela Rojas, Abdulwahab Albassam, Aidan G. Leung, Jett Ngo, Ryan Luo, Peter R. Quawas, Junpyung Kim, Kangkai Liang, Mansi Nanavati, Jonathan Mai, Meng-Chi Tsai, Yun-Tong Tsai, Yize Chen, and Yuanyuan Shi. Llms and agentic ai systems for smart grids: A tutorial on architectures and applications. *arXiv:2607.18147*, 2026.

Yifan Su, Zhaojian Wang, Ming Cao, Mengshuo Jia, and Feng Liu. Convergence analysis of dual decomposition algorithm in distributed optimization: Asynchrony and inexactness. *arXiv:2103.02784*, 2021.

Leon Thurner, Alexander Scheidler, Florian Schäfer, Jan-Hendrik Menke, Julian Dollichon, Frieder Meier, Steffen Meinecke, and Martin Braun. pandapower — an open-source python tool for convenient modeling, analysis, and optimization of electric power systems. In *IEEE Transactions on Power Systems*, volume 33, pages 6510–6521, 2018. doi: 10.1109/TPWRS.2018.2829021.

José R. Vázquez-Canteli, Jérôme Kämpf, Gregor Henze, and Zoltán Nagy. Citylearn v1.0: An openai gym environment for demand response with deep reinforcement learning. *Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Built Areas (BuildSys)*, pages 176–179, 2020. doi: 10.1145/3408308.3427976.

Vassilios Yfantis, Achim Wagner, and Martin Ruskowski. Numerical benchmarking of dual decomposition-based optimization algorithms for distributed model predictive control. *Results in Control and Optimization*, 17:1–19, 2024.

Xiyuan Zhou, Huan Zhao, Yuheng Cheng, Gaoqi Liang, Guolong Liu, Wenxuan Liu, Yan Xu, and Junhua Zhao. Elecbench: A power grid dispatch evaluation benchmark for large language models. In *2025 IEEE Power & Energy Society General Meeting (PESGM)*, 2025. doi: 10.1109/PESGM52009.2025.11225826.

Zhi Zhou, Fang Zhao, Jie Wang, and Yan Liu. Deep reinforcement learning for demand response in microgrids: A review. In *ACM e-Energy*, 2022.